GRAPH NEURAL NETWORKS

图神经网络

课程讲义

图的数学表示与结构

传统图机器学习

VON

课程参考：2026 秋图神经网络 Guojie Song

目录

符号约定 i

第 1 章 图的数学表示与结构 **1**

1.1 预测对象与建图方式 1
1.2 矩阵、邻接表与邻居聚合 3
1.3 可达性、距离与游走 5
1.4 度分布、闭合关系与图信号 8
1.5 节点编号与置换对称性 9
1.6 从固定特征到可学习表示 10

第 2 章 传统图机器学习 **11**

2.1 特征映射与学习流程 11
2.2 节点的局部形状与全局位置 11
2.3 Graphlet：把局部结构变成计数向量 14
2.4 链路预测：局部邻域如何给出证据 15
2.5 Katz：累加不同长度的结构联系 17
2.6 整图特征与 Graphlet 核 19
2.7 WL 颜色细化与图神经网络的联系 21
参考文献 24

符号约定

记号	含义
$G=(V,E)$	图、节点集合、边集合；$n=\lvert V\rvert$，$m=\lvert E\rvert$。
$A\in\mathbb{R}^{n\times n}$	邻接矩阵。有向图约定 A_{uv} 表示从 u 到 v 的边。
$\mathcal{N}(v)$，k_v	节点 v 的邻居集合、度数；无向无权图中 $k_v=\lvert\mathcal{N}(v)\rvert$。
D，L	度矩阵 $D=\mathrm{diag}(k_1,\dots,k_n)$；图拉普拉斯矩阵 $L=D-A$。
$X\in\mathbb{R}^{n\times p}$	节点属性矩阵，每一行对应一个节点，p 为属性维度。
$h_v\in\mathbb{R}^d$，$H\in\mathbb{R}^{n\times d}$	节点表示列向量及其批量矩阵，H 的第 v 行为 $h_v^\top$。
$d(u,v)$	最短路距离；不可达时记为 $+\infty$，自身距离为 0。
c_v，C_v，t_v	中心性分数、局部聚类系数、包含节点 v 的三角形数。
$\phi(G)$，$K(G,G')$	图特征向量、图核；$\mathbf{K}$ 表示一组图的核矩阵。
Π，T	节点置换矩阵、随机游走转移矩阵。
$\rho(A)$，β	邻接矩阵的谱半径、Katz 衰减系数。
$c^{(t)}(v)$，H_{WL}	WL 第 t 轮颜色、总迭代轮数。颜色编号不表示数值大小。

除非特别说明，推导针对**无向**、**无权**、**无自环的简单图**。有向图、加权图以及算法中人为加入的自环会单独说明。图中的节点编号只用于索引，通常不具有大小或顺序含义。

第 1 章　图的数学表示与结构

对一篇论文做分类时，标题和摘要是它自身的信息，引用关系则说明它与哪些论文有关。图学习把这两种信息放在同一个模型中：节点携带属性，边规定哪些对象能够发生联系。要理解图神经网络，首先需要弄清这些联系如何写成数学对象，以及不同的图结构究竟意味着什么。

本章先说明预测任务和建图方式，再把图转成矩阵与邻接表。在此基础上讨论距离、连通性和局部结构，最后解释图模型必须满足的对称性。后续算法中的矩阵乘法与邻居聚合，都可以回到这些基本操作来理解。

1.1　预测对象与建图方式

图可以描述同一类对象之间的联系，也可以连接不同类型的对象。社交网络的节点是用户，边可能表示关注；推荐系统的节点包括用户和物品，边可能表示购买。两者虽然都能写成 $G=(V,E)$，一条边的含义却并不相同。**建图本身就是模型假设的一部分**：它决定算法把什么视为有关联的信息。

例如，论文之间可以按引用关系连边，也可以按文本相似度连边。前者表达知识来源，通常有方向；后者表达内容接近，需要先选择相似度和阈值。如果把这两种关系混为一谈，模型得到的“邻居”就失去了明确含义。

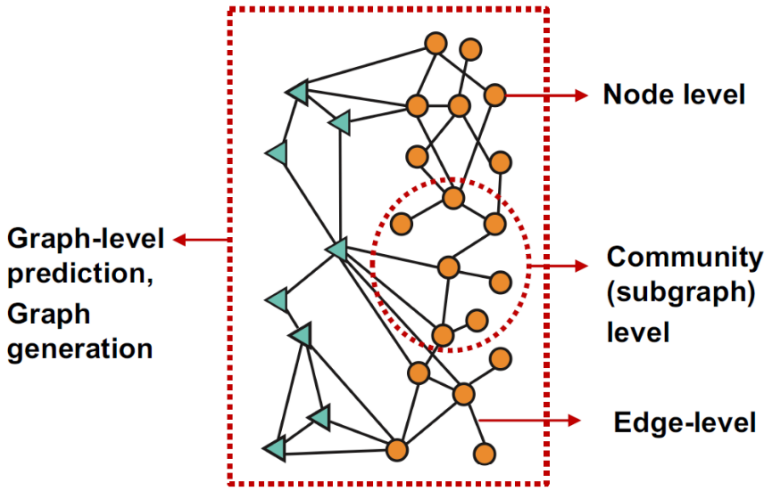


图 1.1　同一张图可以承载不同粒度的任务。节点预测关注一个对象，链路预测关注一对对象，图级预测关注整个关系系统。

任务粒度决定输出形式

设节点属性组成矩阵 $X \in \mathbb{R}^{n\times p}$。节点分类为每个节点输出类别分数，链路预测为节点对输出连边分数，图分类则只输出一组图级分数：

$$F_{\text{node}}(A, X) \in \mathbb{R}^{n\times q}, \tag{1.1}$$

$$s(u, v; A, X) \in \mathbb{R}, \tag{1.2}$$

$$F_{\text{graph}}(A, X) \in \mathbb{R}^{q}. \tag{1.3}$$

这里 q 是类别数。做回归时，输出也可以是一个实数或向量。社区发现寻找内部连接较密的节点集合，不一定事先给定标签；图生成则需要决定节点与边本身，输出对象也会改变。

这些任务共享同一张输入图，但监督信号的位置不同。对节点分类，若只有 $V_{\text{lab}} \subseteq V$ 中的节点有标签，可以写出

$$\mathcal{L}(\theta) = \frac{1}{|V_{\text{lab}}|} \sum_{v\in V_{\text{lab}}} \ell\big(f_\theta(v; A, X), y_v\big) + \lambda\Omega(\theta). \tag{1.4}$$

第一项在有标签节点上计算误差，第二项控制模型复杂度。未标注节点可以在允许的实验设定中参与结构计算，但它们的标签不能进入训练损失。

> **备注 | 传导与归纳**
>
> 传导学习在训练时可以看到待预测节点所在的图结构，任务是补出已有图中的标签。归纳学习要求模型能够处理训练时未见过的节点或整张新图。二者的区别在于测试对象是否已进入训练时可见的结构，不能仅凭“用了监督学习”来判断。数据划分时应同时明确可见的边、属性和标签。

图的类型与关系语义

无向边 $\{u, v\}$ 表示对称关系；有向边 (u, v) 表示从 u 指向 v。权重 w_{uv} 可以表示联系强度，也可以表示距离或成本。这两种含义不能直接互换：在前者中，大权重意味着联系紧密；在最短路问题中，大成本反而意味着不愿经过这条边。

当节点或边具有不同类型时，可以使用异质图。若一对节点之间允许多条关系边，则需要多重图或按关系类型分别记录邻接矩阵。自环 (v, v) 表示节点与自身相连；在神经网络里，自环也常被人为加入，用于让节点在聚合时保留自己的信息。

二部图是一种尤其重要的结构。它的节点集合分成互不相交的 U, W，每条边只跨越两侧。用户与电影构成二部图，而“看过同一部电影的用户彼此连边”会得到用户侧投影。

用 $B \in \{0, 1\}^{|U|\times|W|}$ 记录二部图的跨侧连接，则整张图的邻接矩阵为

$$A = \begin{pmatrix} 0 & B \\ B^{\top} & 0 \end{pmatrix}. \tag{1.5}$$

两个 U 侧节点共享多少个 W 侧邻居，可以直接由内积得到：

$$(BB^{\top})_{ij} = \sum_{w\in W} B_{iw}B_{jw}. \tag{1.6}$$

因此，$BB^{\top}$ 的非对角元素给出带计数权重的投影。若只保留是否共享邻居，就把非零元素变成 1；若要求无自环，还要清空对角线。对角元素原本记录节点的二部图度数。

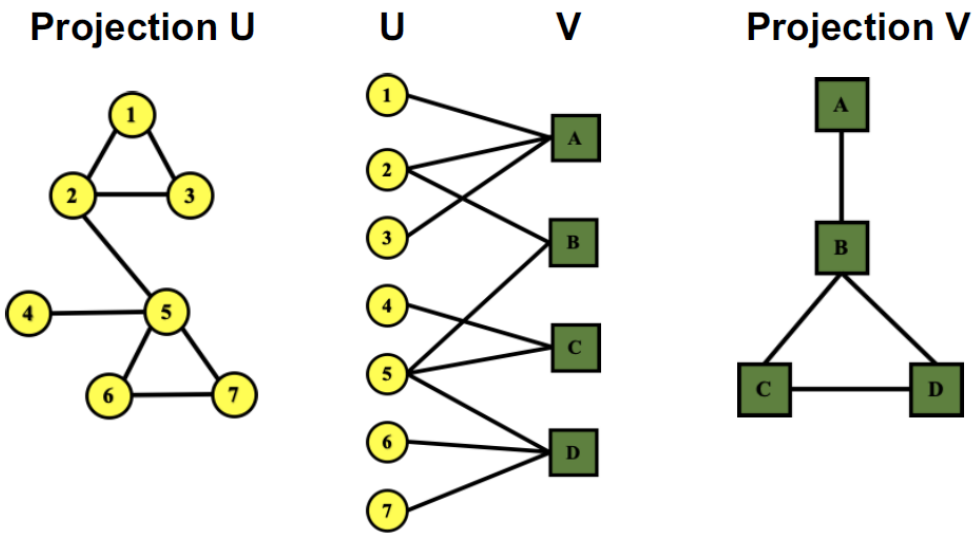


图 1.2 中间是二部图，两侧是分别在 U 与 W 上的投影。共享一个对侧邻居就可能在投影中产生连接。

> **数学直觉 | 投影会放大热门对象**
> 一个有 r 个用户的热门物品，会在用户投影中一次产生至多 $\binom{r}{2}$ 对连接。因而二部图即使稀疏，投影也可能很稠密。投影还会丢失“这些用户究竟共享了哪个物品”的信息；是否保留原二部结构，应由预测问题决定。

1.2 矩阵、邻接表与邻居聚合

图画出来容易理解，算法却需要能够索引和运算的表示。邻接矩阵便于代数推导，邻接表便于遍历，边列表则便于存储与批量传递消息。它们描述的是同一个关系集合。

用一个小图读懂邻接矩阵

考虑图 1.3。按节点 $1,2,3,4$ 排列，它的边集合为 $E=\{\{1,2\},\{1,4\},\{2,4\},\{3,4\}\}$。规定 $A_{uv}=1$ 当且仅当节点 u,v 相邻，则

$$A=\begin{pmatrix}0&1&0&1\\1&0&0&1\\0&0&0&1\\1&1&1&0\end{pmatrix},\qquad k=A\mathbf{1}=\begin{pmatrix}2\\2\\1\\3\end{pmatrix}. \tag{1.7}$$

第 v 行说明节点 v 的邻居是谁。无向图的两个方向描述同一条边，所以 $A=A^{\top}$。无自环意味着 $A_{vv}=0$，而所有元素之和等于 $2m$，因为每条无向边被记录了两次。

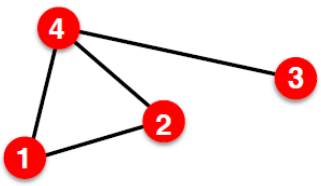


图 1.3 贯穿本章的四节点小图：一个三角形加上连接节点 4 的叶节点 3。

对于有向图，本文始终把行当作起点、列当作终点。因此

$$k_u^{\text{out}} = \sum_v A_{uv}, \qquad k_v^{\text{in}} = \sum_u A_{uv}. \tag{1.8}$$

加权图可以令 $A_{uv} = w_{uv}$，此时行和表示出边权重之和，通常称为强度。不要把它与不考虑权重的邻居个数混用。

矩阵乘法就是沿边汇总信息

假设每个节点携带一个标量 x_v，把它们排成 $x \in \mathbb{R}^n$。矩阵乘法的第 v 个分量为

$$(Ax)_v = \sum_u A_{vu} x_u = \sum_{u \in \mathcal{N}(v)} x_u. \tag{1.9}$$

所以 Ax 把所有节点的邻居求和同时做了一遍。若每个节点携带 p 维属性，把列向量 x_v 转置后逐行放入 X，就得到

$$(AX)_{v,:} = \sum_{u \in \mathcal{N}(v)} X_{u,:}, \qquad AX \in \mathbb{R}^{n \times p}. \tag{1.10}$$

例如在小图中取 $x = (1, 2, 4, 8)^\top$，则 $Ax = (10, 9, 8, 7)^\top$。节点 4 汇总了节点 $1, 2, 3$ 的数值，所以得到 $1 + 2 + 4 = 7$。

单纯求和会让度数大的节点更容易获得较大幅值。令 $D = \operatorname{diag}(k_1, \dots, k_n)$，对于非孤立节点，$D^{-1}AX$ 就把求和改为均值：

$$(D^{-1}AX)_{v,:} = \frac{1}{k_v} \sum_{u \in \mathcal{N}(v)} X_{u,:}. \tag{1.11}$$

孤立节点的 $k_v = 0$，必须另行约定输出为零、保留自身属性，或加入自环。对有向图，按本文的方向约定，AX 汇总出邻居的属性；如果想让消息沿 $u \to v$ 发送并在终点接收，应使用 $A^\top X$。

> **算法实现 | 构造小图并计算邻居均值**
>
> 以下 NumPy 代码只用于小图演示。每条无向边在输入列表中出现一次。

```
import numpy as np
edges = [(0, 1), (0, 3), (1, 3), (2, 3)]
A = np.zeros((4, 4))
for u, v in edges:
    A[u, v] = A[v, u] = 1
X = np.array([[1.], [2.], [4.], [8.]])
degree = A.sum(axis=1, keepdims=True)
mean = (A @ X) / np.maximum(degree, 1)
```

> 这里代码从 0 编号，数学例子从 1 编号；两者的行顺序一致。分母替换保证孤立节点的零行仍输出零。

稀疏图为什么不宜存成稠密矩阵

对 n 个节点，稠密邻接矩阵需要存储 n^2 个数。若 $n = 10^6$，即便每个数只占四字节，也约需 4 TB。许多图的边数远小于 n^2，此时存下大量零没有必要。

表示方式	主要存储内容	适合的操作
邻接矩阵	全部 A_{uv}；稠密存储为 $O(n^2)$	查询某条边、矩阵表达
边列表	m 个节点对及其属性	遍历边、按边传消息
邻接表 / CSR	各节点邻居，约 $O(n+m)$	遍历局部邻域、稀疏乘法

无向图的邻接表通常把每条边存两次。用稀疏结构计算 AX 的代价约为 $O(mp)$，再加上初始化输出等开销；稠密乘法则约为 $O(n^2p)$。后续看图算法的复杂度时，要同时问它按节点遍历、按边遍历，还是枚举节点对。

1.3 可达性、距离与游走

一条边只描述直接关系。实际任务还会问：消息能否从这里传到那里，需要几步，沿途有多少种走法？这些问题分别对应连通性、最短距离和游走计数，它们保留的信息不同。

连通分量与矩阵分块

在无向图中，如果任意两个节点之间都存在路径，图就是连通的。一般图可以划分为若干极大连通子图，称为连通分量。把同一分量的节点放在相邻位置，邻接矩阵就变成分块对角形式：

$$\Pi A \Pi^{\top} = \begin{pmatrix} A_1 & 0 & \cdots \\ 0 & A_2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}. \tag{1.12}$$

其中 Π 只改变节点顺序，A_i 描述一个分量。块间全为零，表示不存在跨分量的边。

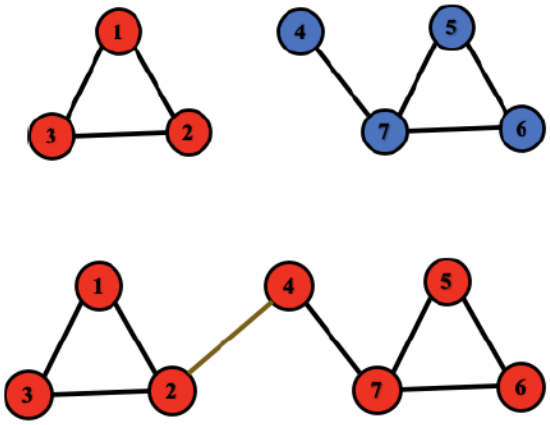


图 1.4 上图的两个连通分量可分别用独立矩阵块表示。下图加入一条跨分量的边后，整张图连通，矩阵也出现跨块非零元素。

有向图要区分方向。若任意两个节点都能沿有向路径相互到达，则称为强连通；若忽略方向后连通，则称为弱连通。强连通分量中的每个节点可以到达其余节点，来自分量外部的单向入口不一定能从内部返回。

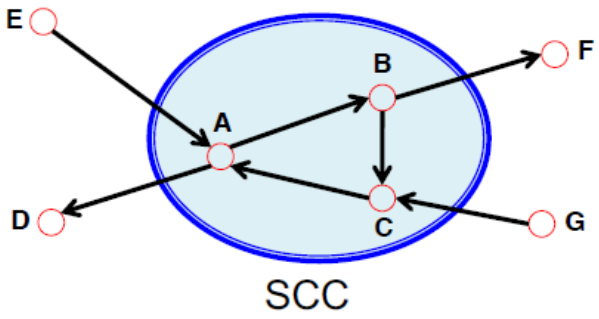


图 1.5 节点 A, B, C 构成强连通分量。E 可以进入该分量，而 D, F, G 可以从该分量到达；这些单向联系不构成相互可达。

最短距离只记录最少步数

本文把允许重复经过节点和边的序列称为**游走**，把不重复顶点的序列称为路径。无权图的距离 $d(u, v)$ 是连接两点的最短路径长度，也等于连接两点的最短游走长度。不可达时距离为 $+\infty$。

在有向图中，路径必须顺着箭头走，所以 $d(u, v)$ 与 $d(v, u)$ 可以不同。边带非负成本时，路径长度改为沿途成本之和；这时不能直接把边数当作距离。

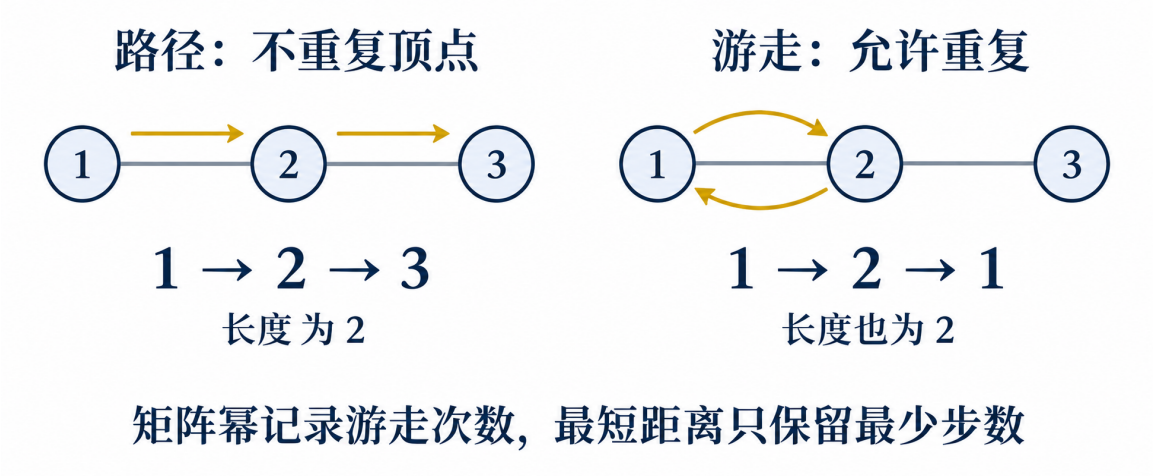


图 1.6 路径与游走的区别。重复节点对计数问题很重要：在 $1 \to 2 \to 1$ 中走了两步，但起点和终点仍然是同一个节点。

对连通无向图，直径与平均路径长度分别为

$$\operatorname{diam}(G) = \max_{u,v} d(u, v), \qquad \bar{d} = \frac{2}{n(n-1)} \sum_{u<v} d(u, v). \tag{1.13}$$

直径关注最远的一对节点，平均距离关注所有节点对的典型分离程度。不连通图若只在可达节点对上求平均，必须相应改变分母；如果只研究最大连通分量，也应明确统计对象。

邻接矩阵的幂为什么能计数

先展开平方矩阵的一个元素：

$$(A^2)_{uv} = \sum_{w} A_{uw} A_{wv}. \tag{1.14}$$

每一项对应一个中间节点 w。只有当 $u \to w$ 和 $w \to v$ 都存在时，该项才等于 1。把所有中间节点加起来，就数出了从 u 到 v 的两步游走。

算法实现 | 广度优先搜索

无权图中，BFS 逐层扩展邻居，第一次访问某节点时就得到最短距离。`adj[v]` 是节点 v 的邻居列表。

```
from collections import deque
def bfs_distances(adj, source):
    dist = {source: 0}
    queue = deque([source])
    while queue:
        v = queue.popleft()
        for u in adj[v]:
            if u not in dist:
                dist[u] = dist[v] + 1
                queue.append(u)
    return dist
```

不在返回字典中的节点不可达。一次 BFS 的时间为 $O(n+m)$；对每个节点都运行一次，则最坏为 $O(n(n+m))$。

同理，若 $(A^{\ell})_{uw}$ 已经数出了到 w 的 ℓ 步游走，再沿 $w \to v$ 走一步，就有

$$(A^{\ell+1})_{uv} = \sum_{w} (A^{\ell})_{uw} A_{wv}. \tag{1.15}$$

由归纳可知，$(A^{\ell})_{uv}$ **是长度恰好为 ℓ 的游走数**。矩阵乘法没有记住历史上去过哪些节点，所以它不会自动排除重复经过。

对式 (1.7) 的小图，

$$A^2 = \begin{pmatrix} 2 & 1 & 1 & 1 \\ 1 & 2 & 1 & 1 \\ 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 3 \end{pmatrix}. \tag{1.16}$$

例如 $(A^2)_{13} = 1$ 对应 $1 \to 4 \to 3$。$(A^2)_{44} = 3$ 则对应经节点 $1, 2, 3$ 分别往返的三条两步游走。因此无向简单图满足 $(A^2)_{vv} = k_v$。

数学直觉 | 距离与计数提供两种信息

对不同节点 u, v，距离是使 $(A^{\ell})_{uv} > 0$ 的最小正整数 ℓ。它只问“最快几步能到”。矩阵幂还保留“走这么多步共有多少种方式”。两对节点的距离可以相同，但游走数量可能相差很大，后者是链路预测中另一种有用信号。

若 A 存储权重，矩阵幂会累加每条游走上权重的乘积，而不是简单的条数。若把 A 归一化为非孤立节点上的转移矩阵 $T = D^{-1}A$，则每一行和为 1，$(T^{\ell})_{uv}$ 表示从 u 出发、ℓ 步后位于 v 的概率。计数与概率具有相似的代数形式，但含义不能混用。

1.4 度分布、闭合关系与图信号

连通性描述能否到达，距离描述相隔多远。要进一步理解一个网络，还需要看连接如何分配，以及邻居之间是否也相互认识。这些统计量既可以描述整张图，也可以成为下一章预测模型的输入。

度数与度分布

随机选取一个节点，其度数为 k 的经验概率为

$$p(k)=\frac{n_k}{n},\qquad \sum_k p(k)=1,\qquad \langle k\rangle=\sum_k k\,p(k)=\frac{2m}{n}. \tag{1.17}$$

这里 n_k 是度数为 k 的节点个数。平均度相同的两张图，连接分配可能完全不同：一张图的各节点度数接近，另一张图则可能由少数高度节点连接许多低度节点。只看平均数会把这种差异抹掉。

在四节点小图中，$p(1)=1/4$、$p(2)=1/2$、$p(3)=1/4$，平均度为 2。度分布不会保存“哪些高度节点彼此相连”，因此也不能唯一确定整张图。即使实测度分布尾部很长，也不能仅凭双对数图近似直线就断定它服从幂律。

聚类系数是条件下的闭合比例

节点 v 有 k_v 个邻居，它们之间至多可能有 $\binom{k_v}{2}$ 条边。设其中实际存在 t_v 条边，每一条都与 v 构成一个三角形。局部聚类系数定义为

$$C_v=\frac{t_v}{\binom{k_v}{2}}=\frac{2t_v}{k_v(k_v-1)},\qquad k_v\ge 2. \tag{1.18}$$

C_v 可以理解为：随机挑选 v 的两个不同邻居，它们也直接相连的比例。度数回答“认识多少人”，聚类系数回答“这些人彼此熟不熟”。

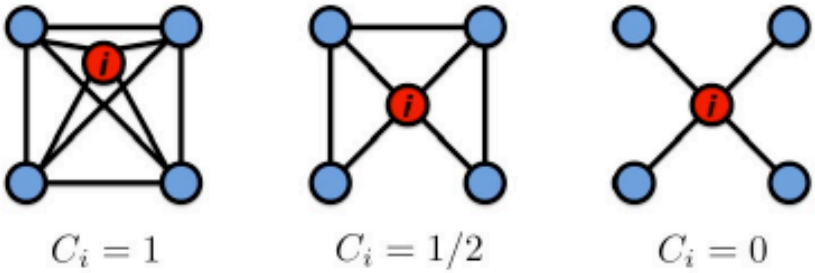


图 1.7 三个中心节点的度数都为 4。邻居间分别有 $6,3,0$ 条边，所以局部聚类系数依次为 $1,1/2,0$。

通常把 $k_v<2$ 的节点的 C_v 约定为 0。小图中四个节点的聚类系数为 $(1,1,0,1/3)$，因此平均局部聚类系数为 $7/12$。另一个常用量是全局传递性：

$$C_{\text{trans}}=\frac{3\,\#\{\text{三角形}\}}{\sum_v\binom{k_v}{2}}. \tag{1.19}$$

分母数的是以不同节点为中心的无序邻居对，一个三角形贡献三对。小图中只有一个三角形，分母为 $1+1+0+3=5$，所以传递性为 $3/5$。它与 $7/12$ 不同，是因为两种平均给节点分配的权重不同。

三角形也可以由矩阵幂得到。每个三角形经过 v 有两个方向的三步闭合游走，因此

$$t_v=\frac{(A^3)_{vv}}{2},\qquad \#\{\text{三角形}\}=\frac{\operatorname{tr}(A^3)}{6}. \tag{1.20}$$

这些等式依赖无向、无自环的简单图假设。在大图上，公式适合推导，实际计算通常使用邻居集合交集，而不显式形成稠密 A^3。

图拉普拉斯：把关系写成平滑约束

当节点上放置数值 x_v 时，图变成了承载信号的结构。例如 x_v 可以是一个路口的流量。若相连路口的数值通常接近，就可以用相邻节点的差异衡量信号是否平滑。

令 $L = D - A$。把二次型展开可得

$$x^\top L x = \sum_v k_v x_v^2 - \sum_{u,v} A_{uv} x_u x_v \tag{1.21}$$

$$= \frac{1}{2} \sum_{u,v} A_{uv} (x_u - x_v)^2 = \sum_{\{u,v\} \in E} (x_u - x_v)^2. \tag{1.22}$$

最后一个式子按无向边只计一次。每一项都非负，所以 L 是半正定矩阵。若 x 在每个连通分量内为常数，能量就是零；反过来，零能量也要求每条边两端取值相同。

> **数学直觉 | 边规定了什么叫“变化”**
> 一维序列用相邻位置之差描述变化，图信号用相连节点之差描述变化。$x^\top L x$ 小，表示信号沿边变化缓慢。这个假设在同类对象倾向连接时很自然，但在“用户连接物品”或不同类别经常相连的图上，需要重新判断哪些属性应该平滑。

1.5 节点编号与置换对称性

在图像里，一个像素有固定的二维坐标，卷积核的左上角也有明确含义。在一般图里，邻居没有天然的第一、第二顺序。如果把节点重新编号，描述的现实系统没有变化，模型就不应因此改变结论。

设 Π 是一个置换矩阵，每行、每列恰有一个 1。重新编号后，属性行与邻接矩阵的行列必须同步变换：

$$X' = \Pi X, \qquad A' = \Pi A \Pi^\top. \tag{1.23}$$

节点级输出跟随节点移动位置，因此应满足**置换等变性**：

$$F_{\text{node}}(\Pi A \Pi^\top, \Pi X) = \Pi F_{\text{node}}(A, X). \tag{1.24}$$

图级输出描述整张图，重新编号后应保持不变，这称为**置换不变性**：

$$F_{\text{graph}}(\Pi A \Pi^\top, \Pi X) = F_{\text{graph}}(A, X). \tag{1.25}$$

等变允许输出的行随输入重排，不变则要求最终结果本身相同。

邻居求和天然满足这一要求。因为 $\Pi^\top \Pi = I$，所以

$$A'X' = (\Pi A \Pi^\top)(\Pi X) = \Pi A X. \tag{1.26}$$

再对每一行应用同一个线性变换和激活函数，仍保持等变。若最后把所有节点表示求和，则 $\mathbf{1}^\top \Pi H = \mathbf{1}^\top H$，得到不变的图表示。

> **备注 | 对称性与区分能力**
> 不变性保证同一张图换编号后得到相同输出，但不保证不同图一定得到不同输出。所有节点属性求和虽然不变，却可能丢掉全部边信息。好的图表示既要尊重编号对称性，也要保留任务真正需要的结构差异。

1.6 从固定特征到可学习表示

图的表示已经确定，接下来要把它变成可供预测使用的信息。传统方法先设计特征映射 ϕ，例如把节点变成“度数、聚类系数、中心性”的向量，再学习预测器 g_θ：

$$\hat{y}_v = g_\theta\big(\phi(v; G, X)\big). \tag{1.27}$$

此时训练可以改变分类边界，却不能改变特征提取规则。如果两个节点在 ϕ 下完全相同，任何只使用该向量的确定性预测器都无法区分它们。

表示学习把特征提取也纳入优化。图神经网络的一般形式是让节点反复接收邻居的信息：

$$m_v^{(\ell)} = \mathrm{AGG}^{(\ell)}\left(\{h_u^{(\ell)} : u \in \mathcal{N}(v)\}\right), \tag{1.28}$$

$$h_v^{(\ell+1)} = \mathrm{UPDATE}^{(\ell)}\left(h_v^{(\ell)}, m_v^{(\ell)}\right), \qquad h_v^{(0)} = x_v. \tag{1.29}$$

聚合操作需要忽略邻居的排列顺序，例如求和、均值或最大值；更新操作将自身状态与邻域信息结合。每多一层，信息就有机会再沿边传播一步。这里的集合记号实际保留重复元素，不能把两个属性相同的邻居当成一个。

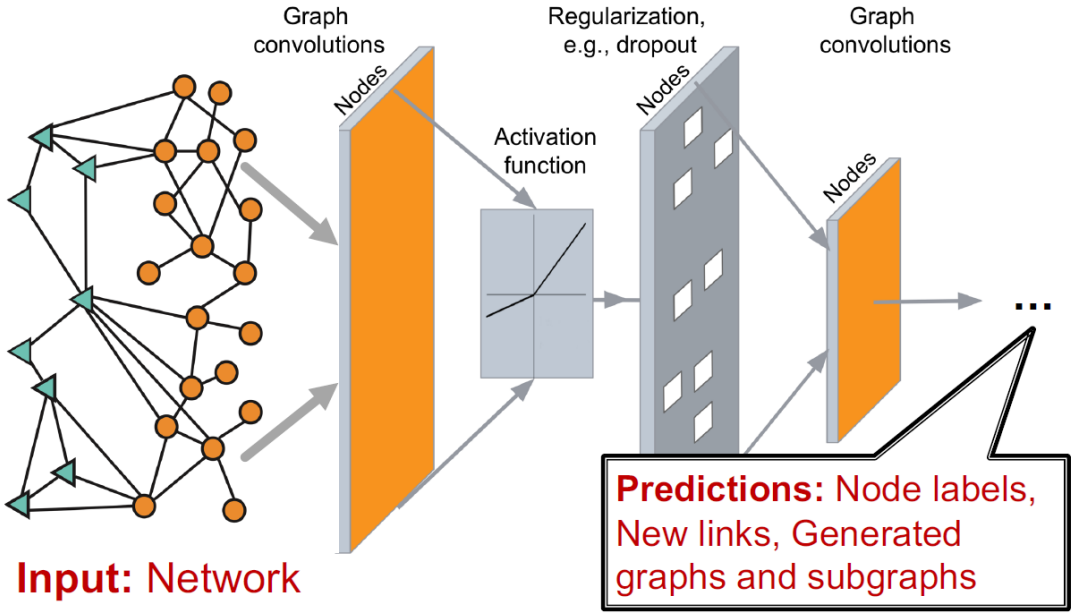


图 1.8 图结构决定信息传播的连接方式，多层可学习变换逐步形成节点表示。最终预测头可以针对节点、链路或整张图。

这个框架说明了图模型的两个独立选择：一是让节点接收哪些结构信息，二是如何把这些信息用于任务。下一章先研究第一种选择的经典答案。理解手工特征保留了什么、遗漏了什么，也就更容易理解可学习聚合的动机。

第 2 章　传统图机器学习

前一章说明图可以提供怎样的信息。本章进一步问：如果暂时不使用图神经网络，如何把这些信息整理成预测模型能接收的向量？核心工作是设计特征映射，它决定模型能“看见”哪些结构。

我们按照预测对象的粒度逐步推进。节点特征描述局部形状和全局位置；链路特征描述两点之间的接近程度；图级特征则比较整张图的结构。算法从计数与最短路发展到矩阵级数，再到迭代颜色细化。最后一种方法与图神经网络的邻居聚合有直接联系。

2.1　特征映射与学习流程

对任意预测对象 z，传统图机器学习通常写成

$$z \xrightarrow{\phi(\cdot;G)} \phi(z;G) \in \mathbb{R}^d \xrightarrow{g_\theta} \hat{y}. \tag{2.1}$$

z 可以是一个节点、一对节点或整张图。第一步由人选择要保留的结构信息，第二步用逻辑回归、SVM、随机森林等模型学习任务规律。特征既可以直接来自原始属性，也可以来自拓扑结构，还可以把两者拼接。

特征设计其实在提出一种相似性假设。度数相近表示两个节点的连接数量相近，中心性相近表示它们在网络中的某种位置相近，graphlet 计数相近则表示局部结构角色相近。这些“相近”并不等价，也不一定都符合当前任务。

> **数学直觉　|　特征是信息的压缩**
>
> 假设 $\phi(z_1;G)=\phi(z_2;G)$，那么只以 ϕ 为输入的模型必有 $g_\theta(\phi(z_1;G))=g_\theta(\phi(z_2;G))$。增加预测器的复杂度，无法恢复已经被特征映射抹掉的信息。因此，判断特征是否合适，应先找出它会把哪些不同结构压成同一个向量。

图数据中的划分还涉及关系可见性。节点分类可以允许未标注节点参与结构计算；链路预测却必须把待预测边从可见图中移除。若把测试边保留在图中再计算度数或共同邻居，特征就可能提前包含答案。训练集上的特征标准化参数也应固定后再用于验证和测试。

2.2　节点的局部形状与全局位置

描述节点时，最容易想到度数和聚类系数。它们计算简单，但只观察很局部的结构。一个节点是否位于两个社群之间，或者是否接近整个网络，需要更广的视野。这就引出了中心性。

度与聚类系数：连接数量和闭合程度

最基础的节点向量可以取 $\phi(v)=(k_v, C_v)^\top$。k_v 反映节点接触多少邻居，C_v 反映邻居间已有连接占可能连接的比例。二者共同区分“认识很多彼此陌生的人”和“处在紧密小圈子中”等情况。

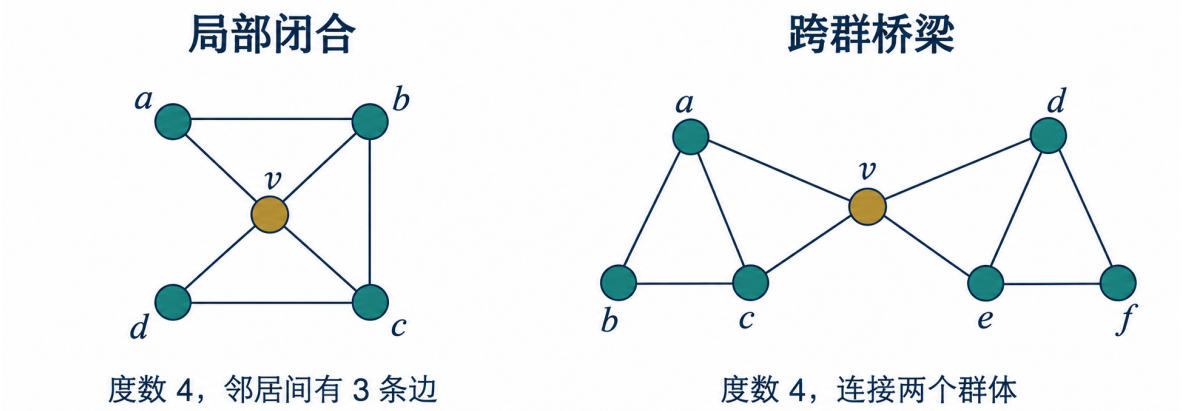


图 2.1 两个高亮节点的度数都是 4。左侧邻域较闭合，右侧节点承担连接两个群体的作用。节点特征需要区分“连接数量”和“结构角色”。

但这个向量仍无法确定节点的全局位置。图 2.1 右侧的 v 一旦移除，左右两组就无法交流；左侧的 v 移除后，其余节点仍沿 $a-b-c-d$ 连通。这个差异需要观察通过节点的路径，不能只看度数。

特征向量中心性：重要邻居的支持

若与重要节点相连也能提高自身的重要性，可以把这种递归关系写成

$$c_v = \frac{1}{\lambda} \sum_{u \in \mathcal{N}(v)} c_u. \tag{2.2}$$

把各节点的关系堆叠起来，正好得到特征值方程

$$Ac = \lambda c. \tag{2.3}$$

λ 用于统一比例，避免递归求和不断放大。对于连通、含至少一条边的无向简单图，非负邻接矩阵的 Perron 特征向量可以取为严格正向量，并在归一化后唯一；对应特征值为谱半径 $\rho(A) > 0$。这里的结论依赖连通性。

用一个有 r 个叶节点的星形图体会这个定义。设中心节点分数为 a，各叶节点分数相同，记为 b。递归方程变成

$$\lambda a = rb, \qquad \lambda b = a. \tag{2.4}$$

于是 $\lambda = \sqrt{r}$，$a/b = \sqrt{r}$。中心节点的分数较高，但比值并不是度数比 r，因为每个叶节点也能获得重要中心节点的支持。

介数中心性：有多少最短路经过这里

把网络中的交流近似为沿最短路传递，一个位于许多最短路中间的节点就可能承担关键中转作用。令 σ_{st} 为 s, t 之间的最短路径总数，$\sigma_{st}(v)$ 为其中把 v 作为内部节点的路径数。无向图的未归一化介数定义为

$$b_v = \sum_{\substack{\{s,t\} \subseteq V \setminus \{v\} \\ s \neq t}} \frac{\sigma_{st}(v)}{\sigma_{st}}. \tag{2.5}$$

算法实现 | 稳定的幂迭代

对连通无向图，可以迭代 $A+I$，其特征向量与 A 相同。加入 I 可避免二部图中直接迭代 A 的周期振荡。

```
def eigen_centrality(A, tol=1e-10, max_iter=1000):
    n = len(A)
    c = np.ones(n) / np.sqrt(n)
    for _ in range(max_iter):
        z = A @ c + c
        z /= np.linalg.norm(z)
        if np.linalg.norm(z - c) < tol:
            return z
        c = z
    raise RuntimeError("iteration did not converge")
```

每步稀疏乘法约为 $O(n+m)$。不连通图可能由谱半径最大的分量主导；多个分量并列时，归一化后的非负解也可能不唯一。

这里对无序节点对只计一次；不可达的节点对贡献为零。分母使一对端点总共只分配一份权重：若有两条同样短的路线，某节点只在其中一条上，就获得 $1/2$。

在链状图 $a-b-c-d$ 中，经过 b 的相关端点对为 $\{a,c\}$ 与 $\{a,d\}$，所以 $b_b=2$；同理 $b_c=2$，两端节点的介数为零。若要把分数归一化到 $[0,1]$，对 $n>2$ 可以除以 $\binom{n-1}{2}$。

备注 | “重要”需要先指定传播假设

度中心性偏重直接连接，特征向量中心性偏重重要邻居的支持，介数中心性偏重最短路中转。如果真实信息并不沿最短路流动，介数就不能直接解释为实际流量。中心性描述的是一个明确的结构视角，而不是无条件成立的价值排名。

接近中心性：到其他节点有多方便

如果关心一个节点向全网传播的距离成本，可以看它到其余节点的平均距离。对连通无向图，常用的接近中心性为

$$q_v=\frac{n-1}{\sum_{u\neq v} d(v,u)}. \tag{2.6}$$

分母越小，分数越大。链状图 $a-b-c-d$ 的距离和分别是 $6,4,4,6$，因此分数为 $1/2,3/4,3/4,1/2$。若省去分子 $n-1$，同一张图内的排序不变，但数值尺度会改变。

不连通图存在无穷距离，直接套用式 (2.6) 会退化。一个自然的替代量是调和中心性：

$$q_v^{\mathrm{harm}}=\frac{1}{n-1}\sum_{u\neq v}\frac{1}{d(v,u)},\qquad \frac{1}{+\infty}=0. \tag{2.7}$$

这让不可达节点贡献零，同时保留到可达节点的距离差别。对于有向图，还应说明关心从 v 出发的距离，还是其他节点到达 v 的距离。

节点的全局中心性通常比局部计数更贵。例如全节点距离可以通过逐点 BFS 获得，无权图最坏为 $O(n(n+m))$。因此在大图上，特征的解释能力还要与计算代价一起考虑。

2.3 Graphlet：把局部结构变成计数向量

聚类系数围绕一种形状工作：三角形。若两个节点的度数和聚类系数都相同，它们仍可能分别处于长链、星形或四节点环中。要把这些差异编码成特征，可以统计节点参与了多少种小图结构。

诱导子图与节点角色

给定节点子集 $S \subseteq V$，诱导子图 $G[S]$ 保留 S 中的节点，以及原图在这些节点之间的**所有边**。节点选定后，不能为了匹配某个形状再删掉额外的边。因此，三角形的三个节点诱导出三角形，不能同时当作一条三节点链。

节点级 graphlet 通常指小型、连通、非同构的诱导子图。对节点特征，仅知道它出现在哪种小图中还不够，还需要知道它位于什么位置。例如三节点链 $a-b-c$ 的两端在结构上等价，但中间节点 b 扮演不同角色。

用保持边关系的节点重命名，也就是图自同构，把能够互换的位置归为同一类。这种位置类别称为**自同构轨道**。固定轨道 j 后，令

$$g_j(v) = \#\{S \subseteq V : v \in S,\ G[S] \text{ 属于相应 graphlet，且} v \text{ 位于轨道} j\}. \tag{2.8}$$

所有轨道的计数组成 graphlet degree vector，简称 GDV。它把普通“度数”扩展为“参与各种局部结构的次数”。

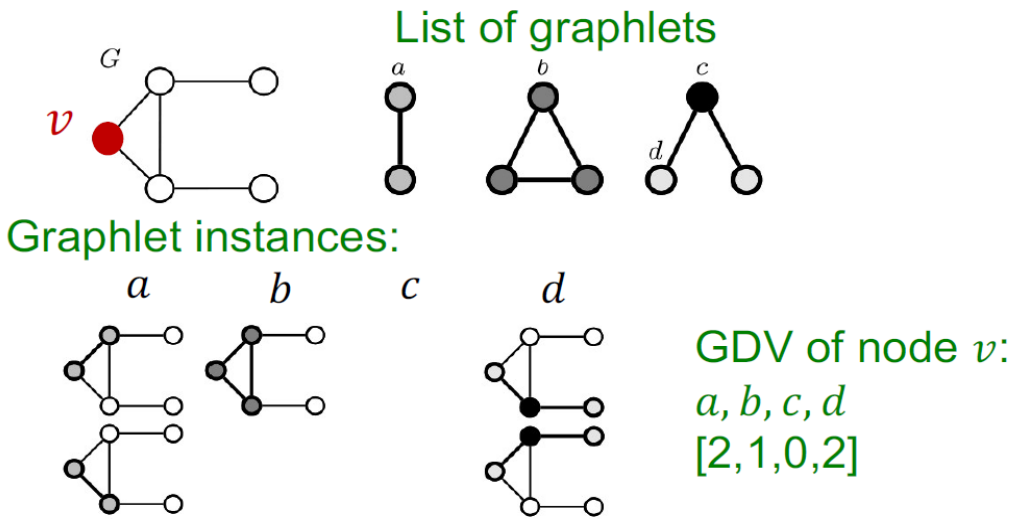


图 2.2 围绕红色节点 v 统计角色。按“边端点、三角形节点、三节点链中心、三节点链端点”的顺序，得到 $(2,1,0,2)$。诱导子图条件使三角形不能再计为开口的三节点链。

图 2.2 中，v 有两个邻居，因此接触两条边；这两个邻居相连，因此 v 在一个三角形中。也正因为邻居相连，v 无法在所选三个节点的诱导子图中充当开口链的中心。沿两个邻居各自连接的外侧叶节点，v 又能分别充当一次链端点。

更丰富的形状带来什么

考虑所有 2 到 5 个节点的连通 graphlet，一共有 30 种形状和 73 个节点轨道，因此得到 73 维 GDV[2]。维度对应的是轨道数，而不是 graphlet 种类数。

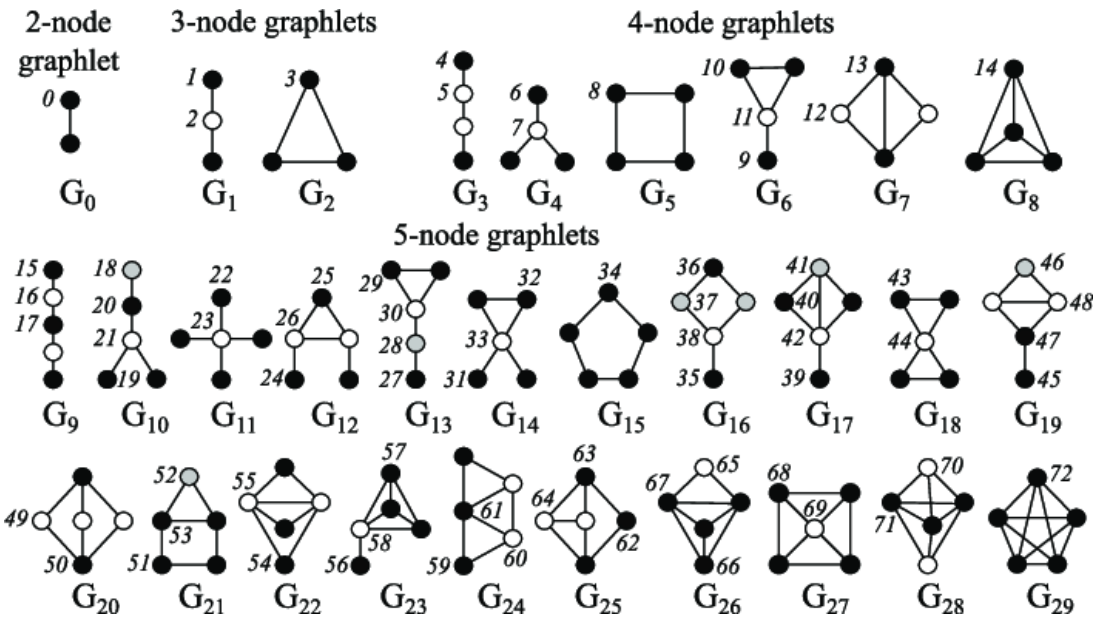


图 2.3 小型连通 graphlet 及其节点轨道。相同形状中的对称位置属于同一轨道，轨道编号为 0 到 72。无需逐一记忆，关键是理解“形状与位置共同决定计数坐标”。

> **数学直觉 | 结构角色相似可以跨越很远的距离**
>
> 两名用户可能位于完全不同的社群，却都承担星形中心或桥接位置，因而具有相近的 GDV。反之，两名彼此相连的用户可能一个是叶节点、另一个是枢纽。结构角色相似与空间接近是两种不同的关系。

GDV 的代价是需要识别和统计小图。形状越大，候选节点集合越多。一个五节点连通 graphlet 最多延伸到根节点的四跳范围，但 GDV 并不完整编码整个四跳诱导邻域；更不意味着计数相同就一定结构同构。它仍然是一种有限维的摘要。

2.4 链路预测：局部邻域如何给出证据

节点分类为一个对象预测属性，链路预测则为一对对象预测关系。以社交网络为例，问题不再是“这个人是什么类型”，而是“这两个人是否可能认识”。因此，输入特征必须同时描述两个端点。

候选节点对与评价

对观测图 $G_{\text{obs}} = (V, E_{\text{obs}})$，无向无自环情形下的候选集合为

$$\mathcal{C} = \{\{u, v\} : u \neq v,\ \{u, v\} \notin E_{\text{obs}}\}. \tag{2.9}$$

算法给每个候选对打分 $s(u, v)$，按分数降序排列。随机缺边恢复会先隐藏一部分已知边；时间预测则用截止时刻之前的图预测后来出现的边。二者回答的问题不同，不宜混用同一种划分来代表另一种能力。

若真实新增边集合为 E_{new}，排序前 K 个候选对为 L_K，那么

$$\text{Precision@}\,K = \frac{|L_K \cap E_{\text{new}}|}{K}, \qquad \text{Recall@}\,K = \frac{|L_K \cap E_{\text{new}}|}{|E_{\text{new}}|}. \tag{2.10}$$

Precision 衡量推荐出的候选有多少正确，Recall 衡量真实新边被找回多少。大量未观测节点对未必都是真实负例，负采样方式和候选集合会影响评价结果。

共同邻居与归一化重叠

最短距离是一种直接特征，但只保留最短的步数。图 2.4 中，(B,H)、(B,E) 与 (A,B) 的距离都是 2，它们的共同邻居数量却不同。

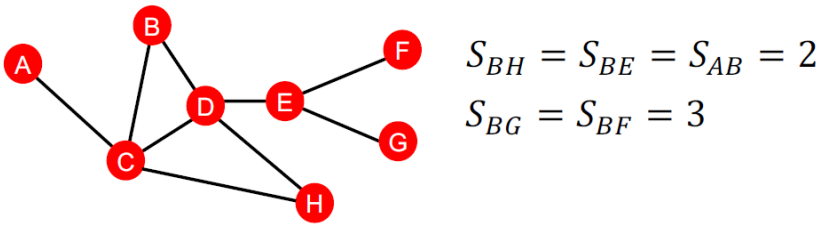


图 2.4 B,H 有两个共同邻居 C,D；B,E 只有共同邻居 D；A,B 只有共同邻居 C。相同的最短距离不能表达这种重叠差异。

共同邻居分数定义为

$$s_{\mathrm{CN}}(u,v) = |\mathcal{N}(u) \cap \mathcal{N}(v)| = (A^2)_{uv}. \tag{2.11}$$

它的假设是“共享的联系人越多，形成关系的机会越大”。但高度节点天然容易共享更多邻居，原始计数没有控制邻域规模。Jaccard 系数改为交集占并集的比例：

$$s_{\mathrm{J}}(u,v) = \frac{|\mathcal{N}(u) \cap \mathcal{N}(v)|}{|\mathcal{N}(u) \cup \mathcal{N}(v)|} = \frac{s_{\mathrm{CN}}(u,v)}{k_u + k_v - s_{\mathrm{CN}}(u,v)}. \tag{2.12}$$

并集为空时可以约定分数为零。它只衡量邻域集合的相似程度，不能直接解释为经过校准的连边概率。

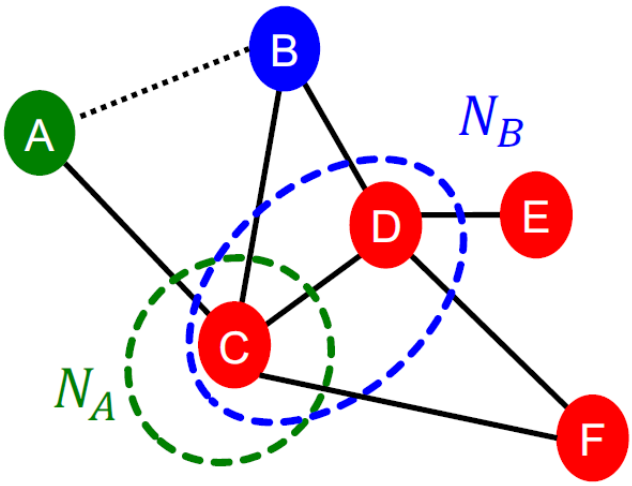


图 2.5 虚线 AB 为待预测边，不计入邻域。$\mathcal{N}(A) = \{C\}$，$\mathcal{N}(B) = \{C, D\}$，因此 CN 为 1、Jaccard 为 $1/2$。

Adamic–Adar 分数进一步区分共同邻居的贡献：

$$s_{\mathrm{AA}}(u,v) = \sum_{w \in \mathcal{N}(u) \cap \mathcal{N}(v)} \frac{1}{\log k_w}. \tag{2.13}$$

低度的共同邻居贡献较大，高度的共同邻居贡献较小。对于不同端点 $u \neq v$，一个共同邻居至少连接它们两个，所以在当前简单图假设下 $k_w \geq 2$，不会出现 $\log 1$ 的分母。

> **数学直觉 | 稀有的共同联系更有辨识度**
>
> 两人都关注一个有数百万关注者的账号，提供的关系证据通常有限；如果两人都与一个很小的圈子联系，信息可能更具体。Adamic–Adar 用共同邻居的度数为这种直觉提供一个简单权重。这个假设是否有效，仍取决于边的真实含义。

在图 2.4 中，$\mathcal{N}(B) = \mathcal{N}(H) = \{C, D\}$，所以 $s_{\mathrm{J}}(B, H) = 1$；而 $\mathcal{N}(E) = \{D, F, G\}$，故 $s_{\mathrm{J}}(B, E) = 1/4$。同一批候选对，可以被距离、数量和比例从不同角度排序。

> **算法实现 | 三个局部链路分数**
>
> 邻接表中的邻居先存成集合。输入端点不同，并且不存在自环。

```
import math
def local_scores(adj, u, v):
    common = adj[u] & adj[v]
    union = adj[u] | adj[v]
    cn = len(common)
    jac = cn / len(union) if union else 0.0
    aa = sum(1.0 / math.log(len(adj[w]))
             for w in common)
    return cn, jac, aa
```

> 这些分数可以直接排序，也可以与其他特征拼接后训练分类器。计算前必须使用移除了待预测边的观测图。

局部重叠有一个明确边界：没有共同邻居时，这三个分数都为零。距离三跳、四跳的节点对即使存在许多间接联系，也无法被区分。这正是下一节引入全局信息的动机。

2.5 Katz：累加不同长度的结构联系

共同邻居只观察两步关系。若允许更多步，就能把间接联系纳入评分；但步数越长，通常应给予越小的权重。Katz 分数把每一种长度的游走都加起来：

$$S_{uv} = \sum_{\ell=1}^{\infty} \beta^{\ell} (A^{\ell})_{uv}, \qquad \beta > 0. \tag{2.14}$$

其中 $(A^{\ell})_{uv}$ 统计长度为 ℓ 的游走，β^{ℓ} 衰减较长游走的贡献。在缺边候选对上，第一项为零，第二项恰好是 β^2 倍的共同邻居数，后续项再加入更远的联系。

从无穷求和到线性方程

在矩阵形式下，把不走任何一步对应的单位矩阵补上：

$$I + S = I + \beta A + \beta^2 A^2 + \cdots. \tag{2.15}$$

若级数收敛，乘以 $I - \beta A$ 后各阶项相消，得到

$$S = (I - \beta A)^{-1} - I. \tag{2.16}$$

这相当于标量几何级数 $1+r+r^2+\cdots=1/(1-r)$，但这里的“公比”是矩阵。正确的收敛条件是

$$\boxed{\beta\rho(A)<1.} \tag{2.17}$$

仅要求 $\beta<1$ 并不充分，因为图中的游走数也在随长度增长。

对无向图，将 A 正交分解为 $Q\Lambda Q^\top$，可以看清这个条件：

$$S=Q\,\mathrm{diag}\!\left(\frac{\beta\lambda_i}{1-\beta\lambda_i}\right)Q^\top. \tag{2.18}$$

每个特征方向上都在求标量几何级数，必须满足 $|\beta\lambda_i|<1$。当 β 接近阈值时，主特征方向的权重可能急剧变大，数值计算也会更加敏感。

> **数学直觉 | 惩罚长游走与游走数量增长**
>
> 每条长游走的权重虽然很小，但它们的数量可能很多。Katz 要求衰减速度足以压住这种增长。β 较小时，分数更偏向局部；β 增大时，更长的间接联系开始影响结果。它并不会保证长程信息对每个任务都更有用。

一个可以手算的例子

考虑三节点链 $1-2-3$，其邻接矩阵为

$$A=\begin{pmatrix}0&1&0\\1&0&1\\0&1&0\end{pmatrix},\qquad \rho(A)=\sqrt{2}. \tag{2.19}$$

从 1 到 3 只能走偶数步：两步有一条游走，四步有两条，六步有四条。因此

$$S_{13}=\beta^2+2\beta^4+4\beta^6+\cdots=\frac{\beta^2}{1-2\beta^2},\qquad 0<\beta<\frac{1}{\sqrt{2}}. \tag{2.20}$$

取 $\beta=0.2$ 时，分数约为 0.04348。最短路径只有 $1\to2\to3$ 一条，但可以通过往返产生更长游走，它们也被 Katz 计入。

实现时不必显式求逆

如果只需要一个源节点 u 到所有节点的分数，令 $s_u=S^\top e_u$。由式 (2.16) 可写成

$$(I-\beta A^\top)s_u=\beta A^\top e_u. \tag{2.21}$$

s_u 的第 v 个分量就是 S_{uv}。无向图中 $A=A^\top$，转置可省略；保留转置能使同一写法也遵守有向图的起点到终点约定。

若用有限长度 L_0 近似，可以逐步计算 $z_0=e_u$、$z_\ell=\beta A^\top z_{\ell-1}$，再将 $z_1,\dots,z_{L_0}$ 相加。每一步只需一次稀疏矩阵向量乘法。若 $\beta\,\|A\|<1$，在相容矩阵范数下，整个分数矩阵的尾项满足

$$\left\|S-\sum_{\ell=1}^{L_0}\beta^\ell A^\ell\right\|\le\frac{(\beta\,\|A\|)^{L_0+1}}{1-\beta\,\|A\|}. \tag{2.22}$$

它说明截断长度应结合衰减速度选取。对没有相互连接的两个分量，所有跨分量的矩阵幂元素都为零，因此 Katz 也不会凭空建立联系。

算法实现 | 一个源节点的 Katz 分数

以下是小图的稠密实现。对非负无向图，$\rho(A) \le \max_v k_v$，因而用最大行和选择 β 是一种保守办法。

```
def katz_from_source(A, u):
    max_degree = A.sum(axis=1).max()
    if max_degree == 0:
        return np.zeros(len(A))
    beta = 0.5 / max_degree
    e = np.zeros(len(A))
    e[u] = 1
    M = np.eye(len(A)) - beta * A.T
    return np.linalg.solve(M, beta * A.T @ e)
```

排序时还要排除源节点自身及已观测边。大图应使用稀疏线性求解，或截断级数迭代，避免形成稠密的全节点对分数矩阵。

2.6 整图特征与 Graphlet 核

当预测对象是一整个分子时，模型需要比较不同大小、不同编号的图。把邻接矩阵直接拉平成向量并不合适：维度随节点数改变，而且同一分子换个原子编号，向量也会改变。图级特征必须以与编号无关的方式概括结构。

从结构计数到合法的核

一种方法是像统计词频那样统计图中的局部模式。例如 $\phi_k(G)$ 可以记录各种度数各出现多少次。这种直方图对编号不敏感，但同度数分布的不同图仍会被混淆。接下来可以把“词汇”换成更丰富的小图结构或节点颜色。

给定显式特征向量 $\phi(G)$，定义

$$K(G, G') = \phi(G)^\top \phi(G'). \tag{2.23}$$

它不仅是一个相似度，也是一个合法的半正定核。为了看到原因，取任意图集合 $G_1, \dots, G_N$，令 $\mathbf{K}_{ij} = K(G_i, G_j)$，则对任意 $a \in \mathbb{R}^N$，

$$a^\top \mathbf{K} a = \left\| \sum_{i=1}^{N} a_i \phi(G_i) \right\|^2 \ge 0. \tag{2.24}$$

因此核矩阵的特征值非负，可以有零特征值。**任意一个看起来合理的相似度，并不自动满足半正定性**；由特征内积构造则直接保证了这一点。

核方法可以把训练图之间的内积交给 SVM 等模型，而不必显式维护一个巨大的特征空间。这里讨论的核是“样本间的内积相似度”，与卷积网络中的滤波核不是同一个概念。

整图 Graphlet 计数

固定节点数 k，枚举该大小下的无标号诱导子图类型 $g_1, \dots, g_r$。这里为了表示整张图，可以允许不连通的类型，也不再指定根节点。定义

$$f_i(G) = \#\{S \subseteq V : |S| = k,\ G[S] \cong g_i\}, \qquad f(G) = (f_1(G), \dots, f_r(G))^\top. \tag{2.25}$$

$\cong$ 表示图同构，即能通过重新编号得到相同的边关系。每个大小为 k 的节点子集只计一次，而不是按节点排列重复计数。若列出了所有类型，就有

$$\sum_{i=1}^{r} f_i(G) = \binom{n}{k}. \tag{2.26}$$

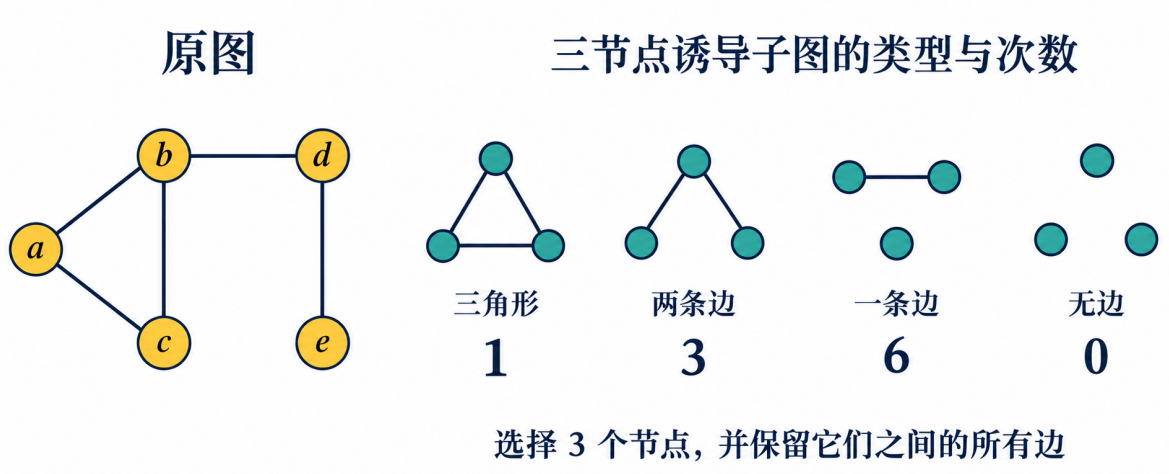


图 2.6 五节点图的三节点诱导子图计数。按三角形、两条边的链、单边加孤立点、三个孤立点的顺序，得到 $f(G) = (1, 3, 6, 0)^\top$，总数为 $\binom{5}{3} = 10$。

在图 2.6 中，若选择三角形的三个节点，必须保留它们之间的三条边。只挑出其中两条边来画成链，会破坏诱导子图的定义，导致重复统计。这个例子也说明为什么节点级 GDV 与整图 graphlet 特征需要分清：前者围绕根节点的角色，后者统计整图中无根的小图类型。

Graphlet 核直接使用 $K(G, G') = f(G)^\top f(G')$。但大图有更多候选节点子集，原始计数可能让相似度主要反映规模。若 $n \geq k$，可以改用频率

$$h(G) = \frac{f(G)}{\sum_i f_i(G)}, \qquad K_{\text{freq}}(G, G') = h(G)^\top h(G'). \tag{2.27}$$

图 2.6 的频率向量是 $(0.1, 0.3, 0.6, 0)^\top$。若图小于 k 个节点，可以把这一组特征设为零，而不能除以零。另一种选择是 ℓ_2 归一化，对非零向量得到余弦核。频率归一化与余弦归一化强调的尺度不同，应按任务选择。

计算成本与采样

直接枚举需要考察 $\binom{n}{k}$ 个节点子集，固定 k 时通常写为 $O(n^k)$ 的枚举量级，此外还要识别小图类型。不能由一般子图同构问题困难，就推断每个固定小 k 的计数都没有更快算法；三角形等特定模式已有专门方法。

对最大度为 Δ 的图，枚举连通 k 节点候选可按邻居扩展，典型上界为 $O(n\Delta^{k-1})$，其中隐藏了与固定 k 有关的常数。这种连通扩展不能直接枚举包含孤立节点的所有类型。

如果只需估计所有 k 节点类型的相对频率，可以均匀采样大小为 k 的节点子集。设采样 M 次，出现类型 i 的次数为 N_i，则 $\hat{h}_i = N_i/M$ 是该类型频率的无偏估计。若改为先选节点再沿边扩展，采样分布会偏向某些结构，必须另行考虑偏差。

Graphlet 保留了有用的局部模式，但随着 k 增大，词汇和计算量都迅速增加。下面的 WL 方法不直接枚举所有小图，而是逐轮利用邻居信息丰富节点描述。

2.7 WL 颜色细化与图神经网络的联系

设想每个节点最初只拿着一张写有类别的卡片。每轮更新时，它先收集邻居的卡片，再把“自己的类别与收到的类别组合”换成一张新卡片。第一轮只能知道邻居的基本情况，第二轮开始知道邻居周围的情况。Weisfeiler–Lehman（WL）颜色细化正是这种迭代过程的离散形式。

颜色是结构签名的编号

无属性图可以给所有节点相同初始颜色 $c^{(0)}(v) = 0$；有离散节点标签时，可以用标签初始化。每一轮构造签名

$$s^{(t+1)}(v) = \left(c^{(t)}(v), \left\{\!\left\{c^{(t)}(u) : u \in \mathcal{N}(v)\right\}\!\right\}\right), \tag{2.28}$$

再压缩成新颜色：

$$c^{(t+1)}(v) = \mathrm{HASH}\big(s^{(t+1)}(v)\big). \tag{2.29}$$

双层花括号表示多重集合：顺序不重要，但出现次数重要。两个蓝色邻居与一个蓝色邻居必须能够被区分。自身颜色单独作为签名的一部分保留，因此旧颜色不同的节点不会在以后被合并。

这里 HASH 是“把不同签名分配不同编号”的压缩映射，不是随意允许碰撞的摘要。实践中可以用完整元组作为字典键。**比较不同图时，必须共享同一轮的颜色字典**；否则两张图里同一个数字可能表示不同的结构。

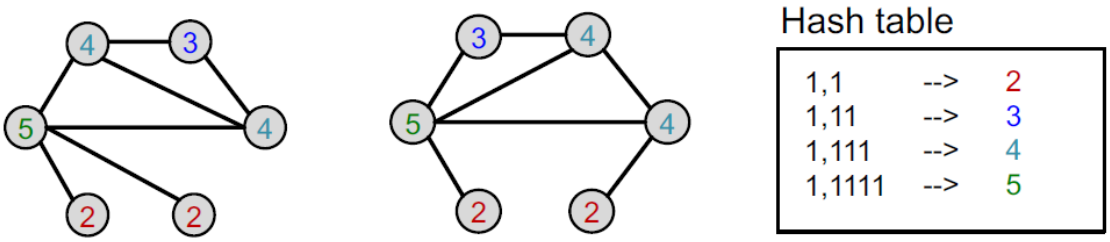


图 2.7 第一轮细化。所有节点初始颜色为 1 时，新签名只由度数决定。两张图具有相同度分布，因而颜色直方图仍相同。右侧字典为两张图共同使用。

图 2.7 中，记号“1, 111”表示自身颜色为 1，并收到三个颜色为 1 的邻居消息。它不是数字一千一百一十一。对应的新颜色为 4，但颜色编号 4 本身没有大小含义。

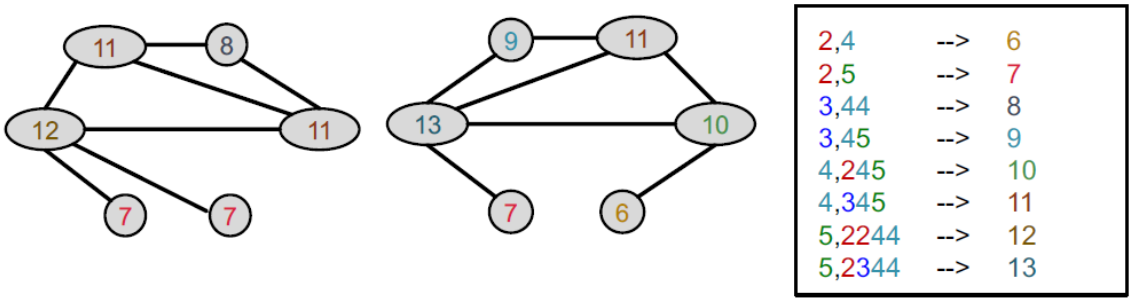


图 2.8 第二轮利用邻居在第一轮的结构类别。左右图中颜色为 5 的节点分别收到多重集合 $\{2, 2, 4, 4\}$ 与 $\{2, 3, 4, 4\}$，因而被分配为不同的新颜色。

第二轮不再只知道“有几个邻居”，还知道“邻居分别有怎样的度数”。若两个同度节点连接了不同类型的邻居，它们便可能分开。继续迭代，信息能够覆盖更远的范围。不过所有节点必须**同步更新**：一轮中的签名只能读取上一轮颜色，不能混入本轮刚更新的值。

将各轮颜色直方图组成图核

令 $\phi_t(G)$ 记录第 t 轮每种颜色出现的次数。WL 子树核把从初始状态到第 H_{WL} 轮的直方图拼接：

$$\phi_{\mathrm{WL}}(G) = [\phi_0(G); \phi_1(G); \cdots; \phi_{H_{\mathrm{WL}}}(G)], \tag{2.30}$$

并定义

$$K_{\mathrm{WL}}(G, G') = \sum_{t=0}^{H_{\mathrm{WL}}} \phi_t(G)^{\top} \phi_t(G'). \tag{2.31}$$

不同轮次应当使用不同的特征坐标，即使编号相同也不能混在一起计数。由于最终仍是特征内积，式 (2.24) 保证其半正定性 [3]。

以四节点链 P_4 和三叶星形图 $K_{1,3}$ 为例。初始颜色相同时，两者都有四个节点，所以初始内积为 $4 \times 4 = 16$。第一轮按度数 $1, 2, 3$ 排列，颜色计数分别为 $(2, 2, 0)$ 与 $(3, 0, 1)$，内积为 6。因此只做一轮细化时，$K_{\mathrm{WL}}(P_4, K_{1,3}) = 22$。这个数是内积相似度，并不是同构判定结果或概率。

算法实现 ｜ 共享字典的 WL 特征

每张图用邻接表表示；下面针对初始颜色相同的无属性图。`Counter` 以“轮次、颜色”为键保存稀疏特征。

```
from collections import Counter
def wl_features(graphs, rounds):
    colors = [{v: 0 for v in G} for G in graphs]
    feats = [Counter({(0, 0): len(G)}) for G in graphs]
    for t in range(1, rounds + 1):
        codebook, updated = {}, []
        for G, c, feat in zip(graphs, colors, feats):
            new = {}
            for v in G:
                key = (c[v], tuple(sorted(c[u] for u in G[v])))
                new[v] = codebook.setdefault(key, len(codebook))
            feat.update((t, label) for label in new.values())
            updated.append(new)
        colors = updated
    return feats

def sparse_dot(f, g):
    return sum(value * g.get(key, 0) for key, value in f.items())
```

不能分别调用后直接比较编号。若训练图与测试图分批处理，应保存各轮签名字典，并一致地扩展新签名。代码中的字典压缩只读取结构，不使用类别标签。

颜色聚合本身每轮遍历邻居一次。采用有效的编码与压缩时，总特征提取代价接近

$$O\big(H_{\mathrm{WL}}(n+m)\big). \tag{2.32}$$

上述易读实现采用排序，每轮还包含约 $O(\sum_v k_v \log k_v)$ 的排序代价。对 N 张图计算全部核值，则有额外的 N^2 对图比较，不能把特征提取的线性代价当作整个训练流程的代价。

颜色相同为什么仍可能结构不同

WL 的颜色是逐轮聚合出的结构签名。它能够区分许多度数相同的节点，但并不完整记录邻域内的每一条边。特别是，它不能普遍识别两个展开分支是否在原图中指向了同一个节点。

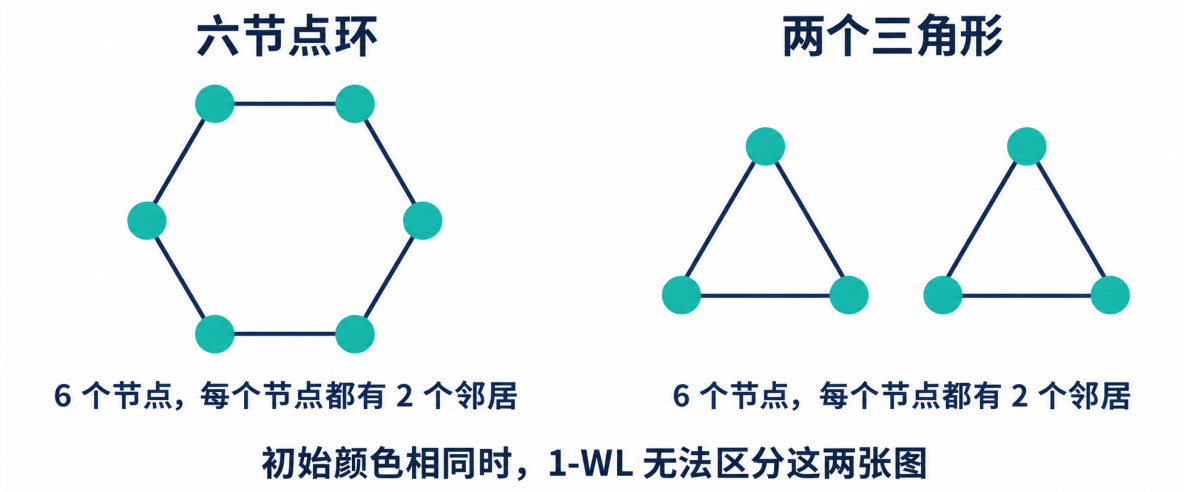


图 2.9 六节点环 C_6 与两个互不连接的三角形。两图不可能同构，但在相同初始颜色下，1-WL 每轮都只能看到“两个同色邻居”。

图 2.9 中，最初所有节点同色。第一轮每个节点都收到两个同色邻居，因此所有节点又得到相同新颜色。这个事实会逐轮重复，两张图每轮的直方图都只有一个非零项，值为 6。然而左图连通且没有三角形，右图不连通且有两个三角形。

> **数学直觉 | 看得更远不等于记住全部结构**
>
> 增加 WL 轮数能够扩大依赖范围，但某些差异在聚合时已经丢失。若每个节点在每一轮收到的多重集合都相同，再多迭代也不能把它们分开。三角形计数能区分这里的两张图，说明不同结构特征之间并不存在简单的“后一种永远更强”。

颜色细化可以持续到节点划分不再发生分裂。稳定指的是划分不变，不要求字典生成的数字编号完全不变。颜色直方图不同可以证明两图不同构；颜色直方图始终相同，**不能证明两图同构**。

离散签名与可学习聚合

回看式 (1.29)，常见消息传递图神经网络与 WL 共享“收集邻居信息，再更新自身表示”的结构。WL 用精确的离散签名和编号，GNN 用向量、可学习变换和连续运算。

例如一个简单的可学习更新为

$$h_v^{(\ell+1)} = \sigma\left(W_{\text{self}}^{(\ell)} h_v^{(\ell)} + W_{\text{nbr}}^{(\ell)} \sum_{u \in \mathcal{N}(v)} h_u^{(\ell)} + b^{(\ell)} \right). \tag{2.33}$$

若 $h_v^{(\ell)} \in \mathbb{R}^{d_\ell}$，则两个权重矩阵均为 $d_{\ell+1} \times d_\ell$，偏置为 $d_{\ell+1}$ 维，输出也为 $d_{\ell+1}$ 维。聚合把数量可变的邻居集合变成固定维向量，而参数共享保证相同规则适用于每个节点。

在相同初始特征、没有额外位置或结构编码的设定下，标准的局部消息传递模型受到 1-WL 区分能力的限制。直观地说，如果两类节点起始状态相同，且每轮收到的状态多重集合也相同，那么任何共享的确定性聚合与更新都会给出相同结果。不同的聚合函数还可能进一步丢失信息，例如相同向量重复出现时，均值不会保留重复次数。

本章的方法因而为学习图模型提供了具体参照：局部统计量保留连接数量与闭合关系，中心性引入全局位置，Katz 汇总多步联系，graphlet 和 WL 则编码局部结构模式。选择哪一种表示，最终取决于任务需要哪些差异，以及允许付出多少计算代价。

方法	主要保留的信息	需要记住的边界
度与聚类系数	邻居数量、局部闭合	不能确定全局位置
中心性	邻居支持、最短路中转或距离	重要性的定义依赖传播假设
CN / Jaccard / AA	二跳重叠及其加权	无共同邻居时全部为零
Katz	衰减后的各阶游走计数	需要 $\beta\rho(A) < 1$
Graphlet	诱导小图形状与角色	随小图规模增加，计数更昂贵
WL	多轮邻居多重集合签名	共享字典；不能完成一般同构判定

检验理解

1. 在星形图中，为什么叶节点的介数为零，却有正的特征向量中心性？两个指标各自回答什么问题？
2. 如果两个节点的共同邻居分数为零，Katz 分数是否一定为零？分别构造一个连通和一个不连通的例子。
3. 为什么只把 WL 的邻居颜色放进普通集合，会把“一个同色邻居”和“多个同色邻居”混淆？
4. 若给每个节点一个唯一 ID 并将其作为初始颜色，比较两张图时可能出现什么问题？这样的结果还能否直接反映不依赖编号的结构相似性？

参考文献

[1] Guojie Song. 图神经网络. 2026 年秋季课程课件：Chapter 1, *Introduction*; Chapter 2, *Traditional Methods for Graph Machine Learning*.

[2] Nataša Pržulj. Biological network comparison using graphlet degree distribution. *Bioinformatics*, 23(2):e177–e183, 2007. 作者公开版本.

[3] Nino Shervashidze, Pascal Schweitzer, Erik Jan van Leeuwen, Kurt Mehlhorn, Karsten M. Borgwardt. Weisfeiler–Lehman Graph Kernels. *Journal of Machine Learning Research*, 12:2539–2561, 2011. 论文与全文.